MOBL-04: Cross-Platform Frameworks

> **Series:** MOBL — Mobile Monitoring | **Notebook:** 4 of 12 | **Created:** February 2026 | **Last Updated:** 10/29/2026

Overview

Cross-platform frameworks like Flutter and React Native let teams ship to iOS and Android from a single codebase. Dynatrace provides dedicated plugins for these frameworks that bridge into the native mobile SDKs, giving you auto-instrumentation, crash reporting, and custom action tracking without writing platform-specific monitoring code. (Session Replay is not available for cross-platform frameworks -- see MOBL-08.)

This notebook walks through instrumenting **Flutter**, **React Native**, **Cordova**, **Xamarin**, and **.NET MAUI** apps with Dynatrace. You will learn how each plugin integrates, how configuration bridging works under the hood, and how to choose the right approach for your stack.

Table of Contents

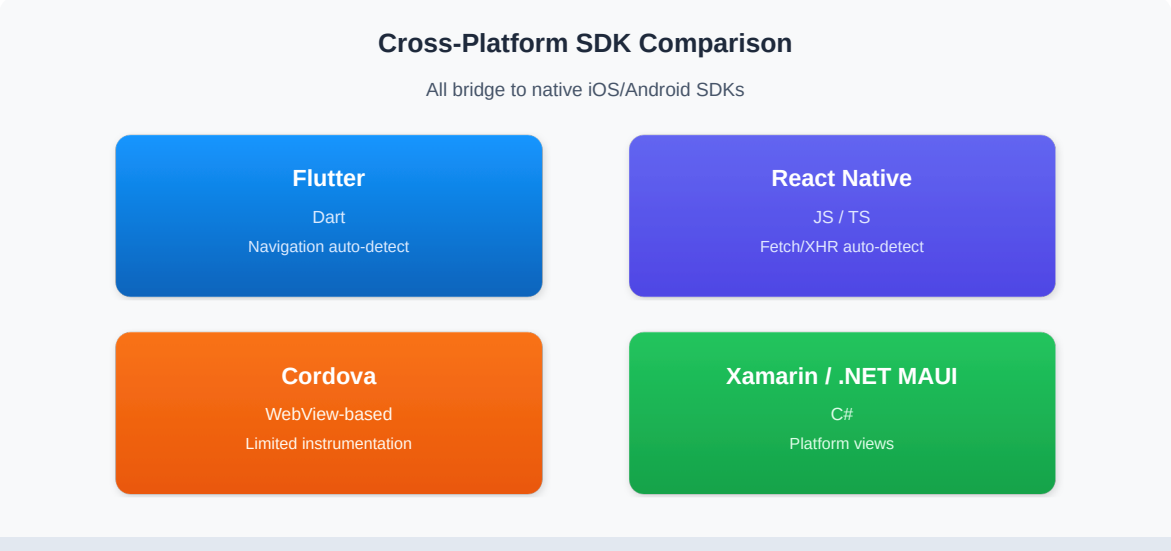
1. SDK Comparison
2. Flutter Setup
3. React Native Setup
4. Configuration Bridging
5. React Native Symbolication
6. Other Frameworks
7. Choosing the Right Approach

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail, with each mobile frontend created in Experience Vitals (RUM Classic frontends: RUM turned on, MOBL-02 §1) — the `user.events` queries here return nothing without it
Permissions	`Mobile app settings` write access; `storage:user.events:read`, `storage:user.sessions:read`, `storage:smartscape:read` for the queries
Flutter	Flutter 3.24.0+ with Dart 3.0.0+ (for Flutter sections)
React Native	React Native 0.72+ with Node.js 18+ (for RN sections)
Mobile App	At least one mobile application configured in Dynatrace
Previous Notebooks	MOBL-01 through MOBL-03 recommended

1. SDK Comparison

Dynatrace offers dedicated plugins for the most popular cross-platform frameworks. Each plugin wraps the native iOS and Android mobile SDKs, exposing framework-specific APIs and auto-instrumentation hooks.



Cross-platform frameworks use platform-specific bridges to the native Dynatrace SDK.

Feature	Flutter	React Native	Cordova	Xamarin
Auto-instrumentation	Navigation, HTTP	Navigation, Fetch/XHR	WebView	Platform views
Crash reporting	Yes	Yes	Yes	Yes
Session replay	Not available	Not available	Not available	Not available
Custom actions	Yes	Yes	Yes	Yes
Maturity	High	High	Moderate	End of support (May 2025); migrate to .NET MAUI

> **Note:** Flutter and React Native are the most actively developed plugins with the broadest feature coverage. Cordova remains supported. **Xamarin is past end of support**: Dynatrace deprecated its Xamarin NuGet package in May 2024 and ended support in May 2025, and recommends the **.NET MAUI** NuGet package instead (§6).

2. Flutter Setup

The Dynatrace Flutter plugin (`dynatrace_flutter_plugin`) integrates into your Flutter project via `pubspec.yaml` and auto-starts based on a configuration file at the project root.

Step 1: Add the Dependency

In `pubspec.yaml`:

```
dependencies:
  dynatrace_flutter_plugin: ^3.349.1   # current release on pub.dev,
10/29/2026
```

Run `flutter pub get` to install the plugin.

Step 2: Get and Apply the Configuration File

Download `dynatrace.config.yaml` from your mobile app's settings in Dynatrace (**Flutter configuration**, or the Frontend creation wizard in the New RUM Experience) and save it in the project root. Then run:

```
dart run dynatrace_flutter_plugin
```

This configures both the Android and iOS projects from `dynatrace.config.yaml`. The file carries each platform's native configuration as a string -- a Gradle `dynatrace { configurations { ... } }` block for Android and Info.plist keys for iOS -- for example (manual-startup variant from the plugin docs):

```
android:
  config: "dynatrace { configurations { defaultConfig { autoStart.enabled
false } } }"
ios:
  config: "&lt;key&gt;DTXAutoStart&lt;/key&gt; &lt;false/&gt;"
```

Step 3: Start the Plugin in `main.dart`

Replace `runApp(MyApp())` with `Dynatrace().start(...)`, and add the navigation observer for automatic view tracking:

```
// main.dart
import 'package:dynatrace_flutter_plugin/dynatrace_flutter_plugin.dart';

void main() =&gt; Dynatrace().start(MyApp());

// in MyApp's build():
MaterialApp(
  home: HomePage(),
  navigatorObservers: [
    DynatraceNavigationObserver(),
  ],
);
```

`Dynatrace().startWithoutWidget(); runApp(MyApp());` is the documented alternative when you cannot wrap the root widget.

> **Correction (09/28/2026).** Earlier revisions kept a plain `runApp()` ("SDK auto-starts based on dynatrace.config file") and showed a `dynatrace: configurations: android: applicationId:` YAML layout. Neither matches the plugin: the app must call `Dynatrace().start(...)`, and the config file holds per-platform `config:` strings.
>
> **Sources:** dynatrace_flutter_plugin (pub.dev) — *"void main() => Dynatrace().start(MyApp());"*
>
> **Tip:** The instrumentation wizard of your frontend shows the Application ID and Beacon URL for each platform. Use exactly what it gives you for the Android and iOS sections.

3. React Native Setup

The Dynatrace React Native plugin (`@dynatrace/react-native-plugin`) provides auto-instrumentation for navigation transitions, Fetch/XHR network calls, and crash reporting.

Step 1: Install and Instrument

```
npm install @dynatrace/react-native-plugin
npx instrumentDynatrace
```

The `instrumentDynatrace` command modifies your native project files (Gradle for Android, Podfile for iOS) to integrate the Dynatrace mobile agent.

Step 2: Configuration File

Create `dynatrace.config.js` at the project root (the instrumentation wizard of your mobile app generates it). The `android.config` string is a Dynatrace Android Gradle plugin block, so the IDs sit inside `autoStart { }` exactly as in MOBL-03 §3:

```
// dynatrace.config.js
module.exports = {
  react: {
    autoStart: true,
    debug: false,
  },
  android: {
    config: `
      dynatrace {
        configurations {
          defaultConfig {
            autoStart {
              applicationId("YOUR_ANDROID_APP_ID")
              beaconUrl("YOUR_BEACON_URL")
            }
          }
        }
      }
    `,
  },
  ios: {
    config: `
      &lt;key&gt;DTXApplicationID&lt;/key&gt;
      &lt;string&gt;YOUR_IOS_APP_ID&lt;/string&gt;
      &lt;key&gt;DTXBeaconURL&lt;/key&gt;
      &lt;string&gt;YOUR_BEACON_URL&lt;/string&gt;
    `,
  },
};
```

Key Configuration Options

Option	Description	Default
`autoStart`	Start monitoring automatically on app launch	`true`
`debug`	Enable verbose logging for troubleshooting	`false`
`lifecycleUpdate`	Also report update cycles on lifecycle actions (creates many more actions)	`false`
`userOptIn`	Privacy mode: user consent must be queried and set before data is collected	`false`

> **Sources:** @dynatrace/react-native-plugin README (npm) — *"lifecycleUpdate boolean false Decide if you want to see update cycles on lifecycle actions as well."*
>
> **Important:** After changing `dynatrace.config.js`, re-run `npx instrumentDynatrace` to apply the updated configuration to native project files.

4. Configuration Bridging

Cross-platform Dynatrace plugins do not implement their own monitoring logic. Instead, they act as **bridges** to the native iOS and Android mobile SDKs.

How Bridging Works

Layer	Component	Role
1	Your cross-platform code (Dart / JavaScript)	Calls the plugin API; framework-level events (navigation, fetch/XHR, Dart/JS errors)
2	Dynatrace plugin (Flutter / React Native / Cordova / .NET MAUI)	Bridges calls and configuration to the native agents
3	OneAgent for iOS / OneAgent for Android	Native instrumentation, crash handling, beacon batching
4	Dynatrace beacon endpoint	Receives beacons; data lands in Grail `user.events` / `user.sessions`

What This Means in Practice

- **Per-platform configuration**: the Android and iOS sections of the config file are applied to different native agents, each with its Application ID and Beacon URL from the instrumentation wizard. Whether both platforms report under one frontend or two depends on how you set them up — the inventory query below shows what you have.
- **Platform-specific behavior**: Some features (like WebView monitoring or specific crash formats) differ between iOS and Android because the underlying native SDK handles them.
- **Native crashes**: Native crashes (Objective-C/Swift on iOS, Java/Kotlin on Android) are captured by the native SDK, not the cross-platform plugin.

- **Dart/JS crashes**: Unhandled exceptions in Dart (Flutter) or JavaScript (React Native) are caught by the plugin and forwarded to the native SDK for reporting.

> **Tip:** Always verify that both platform configurations (Android and iOS) are correct and that both Application IDs appear as active in the Dynatrace UI. A common mistake is configuring only one platform.

Verify Your Mobile App Inventory

Query all configured mobile applications to confirm both Android and iOS entries exist for your cross-platform app:

```
// Full mobile app inventory
// PREFERRED -- Smartscape. Lists every mobile FRONTEND node; a cross-platform
app
// appears once per frontend you created for it.
smartscapeNodes "FRONTEND"
| filter frontend.type == "mobile"
| fields name, id, id_classic, tags
| sort name asc
| limit 50

// CORRECTION (07/30/2026): a previous revision claimed
dt.entity.mobile_application had no
// Grail Smartscape equivalent. It does -- FRONTEND filtered on frontend.type
== "mobile"
// (dt.smartscape.frontend). SaaS 1.344 (07/27/2026, staged tenant rollout
from 07/29/2026)
// makes Smartscape the primary surface for the Digital Experience apps;
verify it has
// reached your tenant. The classic table below still works and remains a real
fallback.
// FALLBACK (classic surface -- still functional):
// fetch dt.entity.mobile_application
// | fields entity.name, id, tags
// | sort entity.name asc
// | limit 50
```

You should see the frontend (or frontends) your cross-platform app reports to. To confirm **both** platforms are sending data, use the `os.name` breakdown in *Platform Distribution* (§6) — if one platform is missing there, revisit that platform's section of the config file.

5. React Native Symbolication

React Native apps ship with minified JavaScript bundles. When a crash or error occurs, the stack trace references obfuscated line and column numbers. **Source map upload** translates these back to your original source code.

How It Works

1. Generate the source map with a **release** build (Hermes and JavaScriptCore are both supported) — see the React Native *debugging release builds* guide.
2. Upload it to Dynatrace: **Settings > Web and mobile monitoring > Source maps and symbol files → React Native → Upload files**. Select the application and platform, enter the JavaScript bundle's file name (Android default `index.android.bundle`, iOS default `main.jsbundle`), and the bundle name and version.
3. When a crash is reported, Dynatrace uses the uploaded source map to symbolicate the JavaScript stack trace.

Upload one source map per platform for every release you ship — a release without one stays minified. Source maps and symbol files share a **1 GiB** storage quota per SaaS environment; older files are deleted automatically when it fills, unless you **pin** the ones you need to keep.

> **Correction (10/02/2026).** Earlier revisions showed a `sourceMap: { android: true, ios: true }` option in `dynatrace.config.js` and a `POST /api/v2/rum/sourcemap` upload endpoint. Neither exists — the plugin README has no such option and points to the symbol-file upload instead.
>
> **Note:** Flutter crashes are symbolicated with the native artifacts — dSYM symbol extract files (iOS) and R8/ProGuard mapping files (Android), see MOBL-03 §6.
>
> **Sources:** Upload and manage symbol files (DT docs) — *"The Dynatrace platform lets you manage Android mapping files, iOS or tvOS symbol extract files and React Native source maps"*, *"For Dynatrace SaaS, the maximum storage size for source maps and symbol files is 1 GiB."*; @dynatrace/react-native-plugin README (npm) — *"Once generated, upload your sourcemaps to Dynatrace."*

Event Volume by Application

Check the volume of mobile events flowing into Dynatrace, grouped by application (`frontend.name`) and event type (`characteristics.classifier`) in the Grail `user.events` store. This helps confirm that your cross-platform instrumentation is actively reporting data:

```
// Event volume by application and event type
fetch user.events, from:-1h
| filter dt.rum.application.type == "mobile"
| summarize event_count = count(), by:{frontend.name,
characteristics.classifier}
| sort event_count desc
| limit 20
```

6. Other Frameworks

Cordova / Ionic

Cordova and Ionic apps run inside a WebView, and the Dynatrace Cordova plugin instruments both the WebView layer and native platform calls.

```
cordova plugin add @dynatrace/cordova-plugin --save
```

Configuration is managed through a `dynatrace.config.js` file similar to the React Native approach. The plugin instruments `XMLHttpRequest` , `fetch` , and page transitions within the WebView.

Xamarin (end of support)

Dynatrace **deprecated** its Xamarin NuGet package (`Dynatrace.OneAgent.Xamarin`) in May 2024, following Microsoft's end of support for the Xamarin SDKs, and **ended support in May 2025**. Existing Xamarin apps should be migrated to .NET and the Dynatrace .NET MAUI package.

.NET MAUI

.NET MAUI is the successor to Xamarin.Forms and is supported:

1. Create a mobile app in Dynatrace, open **Instrumentation wizard > .NET MAUI**, and download `dynatrace.config.json`.
2. Add the `Dynatrace.OneAgent.MAUI` NuGet package (from nuget.org) to the native projects of your app.
3. Add `dynatrace.config.json` to your project as described on the MAUI page.

```
dotnet add package Dynatrace.OneAgent.MAUI
```

> **Correction (09/28/2026).** Earlier revisions named the package `Dynatrace.Xamarin` (not a published package; nuget.org returns 404) and described Xamarin as supported and .NET MAUI as "preview". Both statuses were inverted.
>
> **Sources:** Xamarin NuGet package (DT docs) — *"we will end support for the Dynatrace Xamarin NuGet package in May 2025."*, .NET MAUI (DT docs).

Platform Distribution

Understand which operating systems your mobile users are on. `user.sessions` holds one record per session, so `count()` is a session count. This is especially useful for cross-platform apps to see the iOS-to-Android ratio:

```
// Platform distribution across mobile apps -- one user.sessions record per session
fetch user.sessions, from:-24h
| filter dt.rum.application.type == "mobile"
| summarize session_count = count(), by:{os.name}
| sort session_count desc
```

7. Choosing the Right Approach

Use the following decision matrix to determine the best instrumentation path for your cross-platform project:

Criteria	Flutter	React Native	Cordova	Xamarin / MAUI
New project (2025+)	Recommended	Recommended	Not recommended	.NET MAUI only
Existing project	If already Flutter	If already RN	If already Cordova	Xamarin: migrate to .NET MAUI
Session replay needed	Not available	Not available	Not available	Not available -- Session Replay is native iOS/Android only (MOBL-08)
Team expertise	Dart	JavaScript/TypeScript	Web technologies	C# / .NET
Plugin maturity	Stable, active	Stable, active	Stable, less active	.NET MAUI supported; Xamarin end of support (May 2025)
Auto-instrumentation depth	Deep	Deep	WebView only	Moderate

Recommendations

- **Starting a new cross-platform project?** Choose Flutter or React Native. Both have first-class Dynatrace support with the broadest feature coverage.
- **Migrating from Cordova?** Move to React Native for a smoother transition since both use JavaScript. The Dynatrace plugin API is similar.
- **Migrating from Xamarin?** Move to .NET MAUI and the `Dynatrace.OneAgent.MAUI` package; the Xamarin package is past end of support.
- **Need the deepest auto-instrumentation?** Flutter and React Native offer the most comprehensive out-of-the-box instrumentation, including navigation tracking, HTTP call

monitoring, and crash reporting without manual code changes.

Summary

In this notebook you learned:

- **SDK landscape**: Flutter and React Native plugins are the most mature; Cordova remains supported; Xamarin is past end of support (migrate to .NET MAUI).
- **Flutter integration**: Add the `dynatrace_flutter_plugin` dependency, apply `dynatrace.config.yaml` with `dart run dynatrace_flutter_plugin`, and start with `Dynatrace().start(MyApp())`.
- **React Native integration**: Install `@dynatrace/react-native-plugin`, run `npx instrumentDynatrace`, and configure via `dynatrace.config.js`.
- **Configuration bridging**: Cross-platform plugins delegate to native iOS/Android SDKs. Each platform needs its own Application ID.
- **Symbolication**: React Native requires source map upload for readable crash stack traces.
- **Other frameworks**: Cordova and .NET MAUI are supported; Xamarin reached end of support in May 2025.
- **Decision matrix**: Choose your framework based on project status, team expertise, and required monitoring depth.

Next Steps

Continue to **MOBL-05** to explore custom user actions, manual instrumentation APIs, and advanced tagging strategies for cross-platform mobile apps.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.